Линейная алгебра

Владимир Базлов

Отчёт по лабораторной работе №3

Цель и задания

Освоить основные операции линейной алгебры в Julia: скалярное и внешнее произведения векторов, решение систем линейных уравнений, диагонализацию матриц, вычисление степеней и квадратных корней матриц, а также исследование продуктивности линейных экономических моделей.

Скалярное произведение вектора

[162]:
```
using LinearAlgebra

v = [1, 2, 3]

dot_v = dot(v,v)

println("dot_v = ", dot_v)
```

```
dot_v = 14
```

единственное решение: x = 2,5; y = −0,5

[168]:
```
A_a = [1 1; 1 -1]
b_a = [2,3]
x_a = A_a \ b_a
println(x_a)
```

```
[2.5, -0.5]
```

[170]:
```
A_b = [1 1; 2 2]
b_b = [2, 4]
println("rank(A) = ", rank(A_b))
println("rank[A b]) = ", rank(hcat(A_b, b_b)))
```

```
rank(A) = 1
rank[A b]) = 1
```

[172]:
```
A_c =[1 1; 2 2]
b_c = [2, 5]
println("rank(A) = ", rank(A_c))
println("rank[A b]) = ", rank(hcat(A_c, b_c)))
```

```
rank(A) = 1
rank[A b]) = 2
```

[174]:
```
A_d = [1 1; 2 2; 3 3]
b_d = [1, 2, 3]
println("rank(A) = ", rank(A_d))
println("rank[A b]) = ", rank(hcat(A_d, b_d)))
```

```
rank(A) = 1
rank[A b]) = 1
```

CPULimited

[176]:
```
A_e = [1 1; 2 1; 1 -1]
b_e = [2, 1, 3]
println("rank(A) = ", rank(A_e))
println("rank[A b]) = ", rank(hcat(A_e, b_e)))
```

```
rank(A) = 2
rank[A b]) = 3
```

единственное решение: x = −1; y = 3

[178]:
```
A_f = [1 1; 2 1; 3 2]
b_f = [2, 1,3]
x_f = A_f \ b_f
println(x_f)
```

```
[-0.9999999999999999, 2.0000000000000002]
```

бесконечно много решений: при z = t получаем x = (5 + t)/2, y = (−1 − 3t)/2

[180]:
```
A3a = [1 1 1; 1 -1 -2]
b3a = [2, 3]
println("rank(A) = ", rank(A3a))
println("rank[A b]) = ", rank(hcat(A3a, b3a)))
```

```
rank(A) = 2
rank[A b]) = 2
```

единственное решение: x = −0,5; y = 2,5; z = 0

[182]:
```
A3b = [1 1 1; 2 2 -3; 3 1 1]
b3b = [2, 4, 1]
x3b = A3b \b3b
println(x3b)
```

```
[-0.5, 2.5, 0.0]
```

[184]:
```
A3c = [1 1 1; 1 1 2; 2 2 3]
b3c = [1, 0, 1]
println("rank(A) = ", rank(A3c))
println("rank[A b]) = ", rank(hcat(A3c, b3c)))
```

```
rank(A) = 2
rank[A b]) = 2
```

[186]:
```
A3d = [1 1 1; 1 1 2; 2 2 3]
b3d = [1, 0, 0]
println("rank(A) = ", rank(A3d))
println("rank[A b]) = ", rank(hcat(A3d, b3d)))
```

```
rank(A) = 2
rank[A b]) = 3
```

диагональная матрица 1

[188]:
```
using LinearAlgebra
A = [1.0 -2; -2 1]
E = eigen(A)
D = Diagonal(E.values)
V = E.vectors
display(D)
display(V)
display(V * D * inv(V))
```

```
2×2 Diagonal{Float64, Vector{Float64}}:
 -1.0    ⋅
  ⋅     3.0
2×2 Matrix{Float64}:
 -0.707107  -0.707107
 -0.707107   0.707107
2×2 Matrix{Float64}:
  1.0  -2.0
 -2.0   1.0
```

[190]:
```
A = [1.0 -2; -2 3]
E = eigen(A)
D = Diagonal(E.values)
V = E.vectors
display(E.values)
display(D)
display(V * D * inv(V))
```

```
2-element Vector{Float64}:
 -0.2360679774997897
  4.23606797749979
2×2 Diagonal{Float64, Vector{Float64}}:
 -0.236068    ⋅
   ⋅         4.23607
2×2 Matrix{Float64}:
  1.0  -2.0
 -2.0   3.0
```

[192]:
```
A = [1.0 -2 0; -2 1 2; 0 2 0]
E = eigen(A)
D = Diagonal(E.values)
V = E.vectors
display(E.values)
display(D)
display(V * D * inv(V))
```

```
3-element Vector{Float64}:
 -2.1413361156553594
  0.5151380471280724
  3.6261980685272936
3×3 Diagonal{Float64, Vector{Float64}}:
 -2.14134    ⋅         ⋅
   ⋅       0.515138    ⋅
   ⋅         ⋅       3.6262
3×3 Matrix{Float64}:
  1.0          -2.0  -1.11022e-16
 -2.0           1.0   2.0
 -2.22045e-16   2.0   5.88418e-15
```

[194]:
```
A = [1 -2; -2 1]
result_a = A^10
display(result_a)
```

```
2×2 Matrix{Int64}:
  29525  -29524
 -29524   29525
```

Степень и квадратные корни матрицы

[200]:
```
B = [5.0 -2; -2 5]
E = eigen(B)
S = E.vectors * Diagonal(sqrt.(E.values)) * E.vectors'
display(S)
display(S * S)
```

```
2×2 Matrix{Float64}:
  2.1889   -0.45685
 -0.45685   2.1889
2×2 Matrix{Float64}:
  5.0  -2.0
 -2.0   5.0
```

Степень и квадратные корни матрицы

[202]:
```
C = [1.0 -2; -2 1]
E = eigen(C)
display(E.values)
if all(E.values .>= 0)
    S = E.vectors * Diagonal(sqrt.(E.values)) * E.vectors'
    display(S)
else
    println("корень не существует")
end
```

```
2-element Vector{Float64}:
 -1.0
  3.0
корень не существует
```

Треугольная матрица

[206]:
```
using LinearAlgebra
using BenchmarkTools

A5 = [
    140  97  74 168 131;
     97 106  89 131  36;
     74  89 152 144  71;
    168 131 144  54 142;
    131  36  71 142  36
]

# Собственные значения
values = eigvals(Symmetric(A5))

# Диагональная матрица собственных значений
D5 = Diagonal(values)

# Нижнетреугольная матрица
L5 = tril(A5)

display(values)
display(D5)
display(L5)

# Оценка времени вычислений
@btime eigvals(Symmetric($A5))
@btime tril($A5)
```

```
5-element Vector{Float64}:
 -128.49322764802145
  -55.88778455305688
   42.75216727931894
   87.16111477514521
  542.4677301466143
5×5 Diagonal{Float64, Vector{Float64}}:
 -128.493      ⋅         ⋅         ⋅         ⋅
    ⋅      -55.8878     ⋅         ⋅         ⋅
    ⋅         ⋅       42.7522     ⋅         ⋅
    ⋅         ⋅          ⋅       87.1611     ⋅
    ⋅         ⋅          ⋅         ⋅       542.468
5×5 Matrix{Int64}:
 140    0    0    0   0
  97  106    0    0   0
  74   89  152    0   0
 168  131  144   54   0
 131   36   71  142  36
  1.870 μs (13 allocations: 2.44 KiB)
  46.162 ns (2 allocations: 272 bytes)
```

Проверка продуктивности

[208]:
```
using LinearAlgebra

M = [1.0 2; 3 4]

A_a = M
A_b = (1/2) .* M
A_c = (1/10) .* M

function check_by_definition(A)
    B = I - A

    if abs(det(B)) < 1e-10
        println("I - A вырождена: единственного решения нет")
        return
    end

    C = inv(B)
    println("Матрица A:")
    display(A)
    println("(I - A)^(-1):")
    display(C)
    println("Продуктивна: ", all(C .>= -1e-10))
end

check_by_definition(A_a)
check_by_definition(A_b)
check_by_definition(A_c)
```

```
Матрица A:
2×2 Matrix{Float64}:
 1.0  2.0
 3.0  4.0
(I - A)^(-1):
2×2 Matrix{Float64}:
  0.5  -0.333333
 -0.5   0.0
Продуктивна: false
Матрица A:
2×2 Matrix{Float64}:
 0.5  1.0
 1.5  2.0
(I - A)^(-1):
2×2 Matrix{Float64}:
  0.5   -0.5
 -0.75  -0.25
Продуктивна: false
Матрица A:
2×2 Matrix{Float64}:
 0.1  0.2
 0.3  0.4
(I - A)^(-1):
```

Критерий обратимости

[210]:

```
M = [1.0 2; 3 1]

A_a = M
A_b = (1/2) .* M
A_c = (1/10) .* M

function check_inverse_criterion(A)
    B = I - A

    println("A = ")
    display(A)

    if abs(det(B)) < 1e-10
        println("Обратная матрица не существует")
        return
    end

    C = inv(B)
    display(C)

    println("Продуктивна: ", all(C .>= -1e-10))
end

check_inverse_criterion(A_a)
check_inverse_criterion(A_b)
check_inverse_criterion(A_c)
```

```
A =
2×2 Matrix{Float64}:
 1.0  2.0
 3.0  1.0
2×2 Matrix{Float64}:
 -0.0  -0.333333
 -0.5   0.0
Продуктивна: false
A =
2×2 Matrix{Float64}:
 0.5  1.0
 1.5  0.5
2×2 Matrix{Float64}:
 -0.4  -0.8
 -1.2  -0.4
Продуктивна: false
A =
2×2 Matrix{Float64}:
```

Спектральный критерий продуктивности

[212]:
```
function check_spectral_criterion(A)
    values = eigvals(A)

    println("Собственные значения:")
    display(values)

    println("Модули собственных значений:")
    display(abs.(values))

    println("Продуктивна: ", all(abs.(values) .< 1))
end

M = [1.0 2; 3 1]

check_spectral_criterion(M)
check_spectral_criterion((1/2) .* M)
check_spectral_criterion((1/10) .* M)

A_d = [
    0.1 0.2 0.3;
    0.0 0.1 0.2;
    0.0 0.1 0.3
]

check_spectral_criterion(A_d)
```

```
Собственные значения:
2-element Vector{Float64}:
 -1.4494897427831779
  3.4494897427831783
Модули собственных значений:
2-element Vector{Float64}:
 1.4494897427831779
 3.4494897427831783
Продуктивна: false
Собственные значения:
2-element Vector{Float64}:
 -0.7247448713915892
  1.724744871391589
Модули собственных значений:
2-element Vector{Float64}:
 0.7247448713915892
 1.724744871391589
Продуктивна: false
Собственные значения:
2-element Vector{Float64}:
 -0.14494897427831785
  0.34494897427831783
Модули собственных значений:
2-element Vector{Float64}:
 0.14494897427831785
```

Вывод

В ходе лабораторной работы были выполнены основные операции линейной алгебры в Julia. Вычислены скалярное и внешнее произведения вектора; системы линейных уравнений исследованы с помощью рангов матрицы коэффициентов и расширенной матрицы; для симметричных матриц получены собственные значения и диагональные представления; выполнены возведение матрицы в степень, проверка квадратных корней и анализ матрицы